Part 9 — Reading the data and running the loop: integration, sequencing, RNA-seq and the biofoundry (weeks 32 to 34)

Weeks 20 to 31 were about making things. These three are about finding out what you made and whether you can believe the answer: sequencing a strain, counting transcripts, and reading a plate of fluorescence from a robot. The common thread is that each measurement arrives with a bias that is easy to miss and easy to compute — a read-depth threshold, a normalisation assumption, an edge effect — and each week's useful content is the arithmetic that tells you how large the bias is before you act on the result.

A limit on what is in here. This environment has no network access, so I could not run breseq, DESeq2, salmon, the Opentrons simulator or a download from the Sequence Read Archive. Where those tools would produce output, the chapters give you the command, explain what its output means and tell you what to look for, and are marked as not run. Everything quantitative is a first-principles calculation or a simulation I ran, with the parameters printed beside it — the sequencing statistics are analytic, the RNA-seq and plate-effect results come from simulated data with known truth, and the plate plan is code that was executed. That has an advantage for teaching: with simulated data you know the right answer, which is the only way to see how much a threshold or a normalisation costs.

The tracker's problems for these weeks (J1, J2 and the troubleshooting questions) have their own data; I have used constructed examples with the same structure and said so. The methods transfer and the numbers will differ from the tracker's model answers.

Ch.	Week	Chapter
32	32	Genomic integration, sequencing data and wet-lab reality
33	33	RNA-seq and differential expression: what a count matrix assumes
34	34	The biofoundry as a data pipeline: plate plans, protocols and position effects

Genomic integration, sequencing data and wet-lab reality

Industrial strains do not carry pathways on plasmids, and the arithmetic of why is a two-line recursion. The same discipline — compute the failure rate before you trust the result — is what makes a sequencing table or a failed assembly interpretable.

Why integrate: the plasmid-loss recursion

Part 1 chapter 10 said a plasmid does not stay put without selection and burden makes it worse. Put numbers on it. Let p be the fraction of cells carrying the plasmid, l the probability that a division produces a plasmid-free daughter, and s the growth advantage of a plasmid-free cell over a plasmid-bearing one — the burden. After one generation the carrying fraction is

```
p′ = p(1 − l) / [ p(1 − l) + (1 − p + p·l)(1 + s) ]
```

Iterating it, starting from a fully carrying population:

Loss per division	Advantage of plasmid-free cells	Carrying at 40 generations	at 70	Below 50% at
10^{-4}	5%	98.7%	94.2%	—
10^{-3}	2%	94.1%	86.2%	—
10^{-3}	5%	88.5%	60.6%	generation 79
10^{-2}	2%	56.1%	29.3%	generation 46
10^{-2}	5%	37.5%	8.6%	generation 33

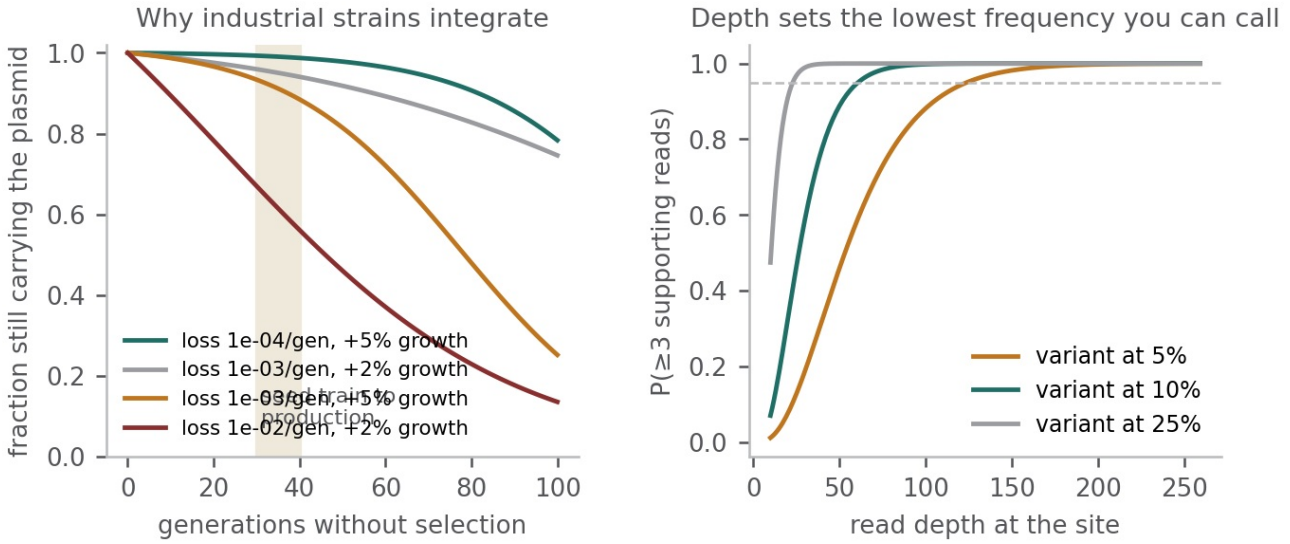


Figure 32.1 Left: the fraction of cells still carrying a plasmid against generations without selection, for four combinations of loss rate and burden; the shaded band is a vial-to-production seed train of thirty to forty generations. Right: probability of at least three reads supporting a variant against depth, at three variant frequencies. Computed for this module; the loss rates and burdens are illustrative, and measuring your own is a plate-count experiment.

Read the third row against the shaded band in the figure. A plasmid that is stable in a bench flask over a few generations can be a majority-free culture by the time it fills a production vessel, because the two rates compound: a loss that is modest per generation is amplified by the growth advantage of the cells that lost it. A seed train from a vial to a 1,000 L vessel is a few tens of generations, which is squarely in the region where a leaky or burdensome plasmid starts to fail. That is the quantitative content of "industrial strains integrate". The other reasons are dosage and antibiotics — a single chromosomal copy is a defined dose, whereas plasmid copy number varies from cell to cell, and no one wants to add an antibiotic to a hundred-cubic-metre fermentation.

Serine integrases and landing pads

Serine integrases such as Bxb1 and φC31 recombine two short, different sites: **attB** on the bacterial side and **attP** on the phage side. The integrase binds both, cleaves each strand, rotates one half of the complex by 180° and religates, producing two hybrid sites, **attL** and **attR**, that flank the inserted DNA. The thing to understand is why this matters for engineering: attL and attR are *not substrates for the integrase alone*. Reversal needs a second protein, a recombination directionality factor, which you do not supply. So integration is effectively irreversible, which buys three things: the integrant does not excise itself during a fermentation, you can use the integrase transiently and lose it, and the integrase will not insert a second copy into the product's own sites. Compare that with a resolvase or a tyrosine recombinase, where the reaction is readily reversible and an insertion can excise.

A **landing pad** is an attB (or attP) site installed in advance at a chosen chromosomal locus, so that every later construct goes to the same place by the same reaction. A safe harbour is a locus where inserting DNA does not disrupt a gene, an essential regulatory element or a transcription–replication conflict — and, an engineering point that is often missed, where the *dosage* suits you. In a fast-growing bacterium replication starts at the origin before the previous round has finished, so genes near the origin are present in more copies than genes near the terminus. For a gene at a fractional position x along the replichore from origin (0) to terminus (1), the copy number is $2^{(C/\tau)(1\,-\,x)}$, with C the replication time and τ the doubling time. With C ≈ 40 minutes (roughly constant at fast growth — an assumption) the origin-to-terminus ratio is 4 at a 20-minute doubling time, 2.5 at 30 minutes and 1.6 at 60. So the same integrated cassette gives two-to-four-fold different dose depending on where it sits and how fast the cells grow, which is worth knowing before you compare two integrants at different loci and attribute the difference to the promoter.

A one-step integration of a four-gene cassette, drawn on paper: a **donor plasmid** carrying attP, the four-gene cassette, and a selectable marker flanked by FRT sites for later removal, on a replicon that cannot replicate in the recipient (a conditional origin that needs a host-supplied initiator protein, so the donor is a dead end unless it integrates); a **helper plasmid**, temperature-sensitive, expressing the integrase; a recipient with attB at the safe-harbour locus. Transform the helper, then the donor; select on the marker; the survivors have integrated, since the donor cannot otherwise be maintained. Verify with two PCRs, one across attL and one across attR, each with one primer in the chromosome and one in the cassette — which also confirms orientation. Express the flippase to excise the marker, leaving one FRT scar; cure the helper by growth at the non-permissive temperature; and sequence the whole cassette, because a long integration is exactly where a mutation or a deletion would hide, as chapter 25's error budget said. Cassettes of five to ten genes have been integrated this way in one step; efficiency falls with size, so the first run for a large cassette is a test of what you actually get.

Editing without a double-strand break

	Recombineering (Red + oligo or cassette)	Cas9 nuclease with a template	Base editing
What it changes	Any defined substitution, deletion or insertion	Any defined edit, enriched by killing the unedited	A single base in a window — C→T or A→G
What it needs	Red proteins; homology arms; an oligo or cassette	Cas9, a guide, a template that destroys the target site	A deaminase–Cas fusion and a guide with a suitable PAM
Failure mode	Low frequency without selection; mismatch repair removes oligo changes	Escapers; a lethal cut if repair fails; plasmid toxicity	Bystander edits in the window; no insertions or deletions; PAM position
Bacterial context	Native to bacteria; works without a break	No efficient end-joining, so an unrepaired cut kills	No break needed, so it avoids the lethality

Prime editing, which dominates attention in the mammalian literature, is a tool that transfers poorly to bacteria, as the tracker says. Bacterial labs instead use base editors for point changes and CRISPR–recombineering for anything else — the combination of chapters 23 and 24. The practical advice is to choose by the edit you need: a point mutation within reach of a PAM, base editing; a deletion or insertion, recombineering with Cas9 counterselection; many loci at once, MAGE.

Sequencing data, from a file to a table

A FASTQ file holds four lines per read: an identifier, the sequence, a plus sign, and a quality string. Each character of the quality string encodes a **Phred score** $Q = -10\,\log_{10}(P_{error})$, stored as the character with ASCII code Q + 33. So `?` (code 63) is Q30, an error probability of 0.1%, and `I` (73) is Q40. A 150-base read with an average Q30 has 0.15 expected errors; at Q20 it has 1.5, which means most reads carry one.

The workflow, which I have not run here: `fastqc reads_1.fastq.gz reads_2.fastq.gz` for the quality report, whose per-base quality plot and adapter-content plot tell you what to trim; align with `bwa mem ref.fasta reads_1.fastq.gz reads_2.fastq.gz > aln.sam`; convert and sort with `samtools sort -o aln.bam aln.sam` and index with `samtools index aln.bam`; view coverage with `samtools depth` or a genome browser. The file formats are worth being able to say: FASTQ holds reads with qualities; SAM is the text alignment, one line per read; BAM is its compressed binary form and the index

(.bai) lets you jump to a region; a reference is FASTA or GenBank, and variants are reported as VCF or, in breseq's case, as an HTML table.

How much depth. For N reads of length L on a genome of size G, the mean depth is c = N·L/G and the fraction of the genome with no reads is e^{-c} (the Lander–Waterman estimate). For a 4.6 Mb genome and 150-base reads, 10^5 reads give 3× depth and leave 3.8% uncovered — 176 kb with no data; 5×10^5 give 16× and leave under one base uncovered; 10^6 give 33× and 2×10^6 give 65×. Coverage is therefore cheap; what depth buys is the ability to call *variants*, and that needs more than coverage.

What depth buys, and what it costs. A variant present in a fraction f of reads is called if at least k reads support it. With k = 3 and binomial sampling, the probability of calling a variant at 10% frequency is 59% at 30× depth, 95% at 60× and essentially certain at 100×; at 5% frequency it is 19% at 30×, 58% at 60×, 88% at 100× and 100% at 200×. Those are the numbers behind figure 32.1's right panel, and they say that for a clonal isolate, where every real variant is at or near 100% frequency, 30× is plenty, whereas for a *population* sample where you want to see subpopulations at a few per cent, you need 100 to 200×.

The cost is false calls. If sequencing errors were independent at 10^{-3} per base, then requiring three reads to agree on the same wrong base would still produce, over 4.6 million sites and three possible wrong bases each, an expected **2.3 false calls at 30× depth, 83 at 100× and 2,100 at 300×**. At 10^{-4} the numbers are 0.002, 0.09 and 2.3. This is an idealised independent-error model, and real errors are not independent — they cluster in particular sequence contexts, which makes some sites worse than the average — but it shows the structure: deeper sequencing finds rarer real variants and also finds more noise, so a caller must also use base quality, strand balance and a minimum frequency. It is why polymorphism-calling modes demand more depth *and* produce more false positives than consensus modes.

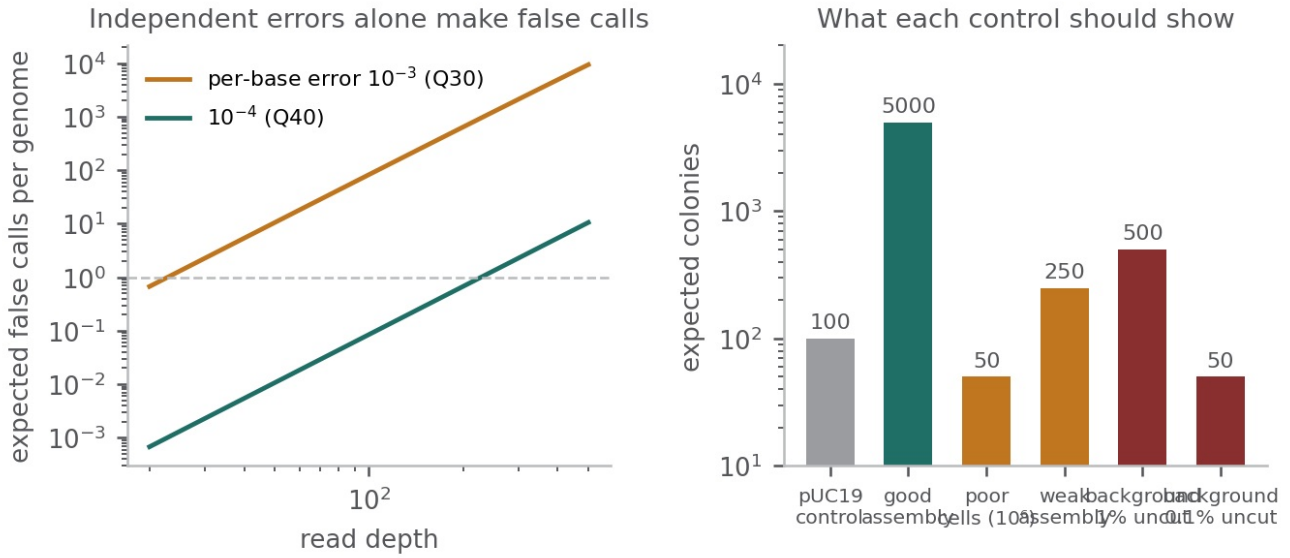


Figure 32.2 Left: expected false variant calls per genome against depth for two per-base error rates, in an independent-error model requiring three supporting reads. Right: the number of colonies each control should give in a transformation, from the assumptions in the text. Computed for this module.

breseq maps reads to a reference and predicts mutations in haploid microbial genomes, including the *new junctions* that betray a mobile-element insertion — reads that span from the reference into an insertion sequence. It has two modes. Consensus mode reports mutations present in essentially all reads, which is right for a clonal isolate. Polymorphism mode (the `-p` option) reports mixed sites down to a few per cent frequency, which is what you want for a population sample from an evolution experiment, and needs the depth above. The tracker's example command is `breseq -j 8 -r REL606.gbk SRR2584524.fastq.gz`, with a note to confirm the accession. The HTML output has three evidence tables: mutation predictions (point mutations and small indels, with the gene, the amino-acid consequence and the frequency), missing coverage (deletions) and unassigned new junction evidence (insertions and rearrangements that breseq saw reads for but could not place) — and the habit to build is to read the third table as well as the first.

To argue which one or two mutations of an evolved strain are causal, use chapter 28's tests: is the same gene or pathway hit in independent lineages; does the mutation fall in a regulator, a transporter or the selected pathway; does a loss-of-function mutation make sense for the selection applied; and then reconstruct each candidate in the parent. A mobile-element insertion into a gene is a gene knockout, and in a long evolution experiment insertion sequences are a common route to loss of function — the point of reading the junction table.

Wet-lab controls, and troubleshooting by arithmetic

Molar ratio. Convert mass to moles with pmol = ng/(0.66 × bp), since a base pair weighs about 660 g/mol: 50 ng of a 5.0 kb vector is 0.0152 pmol, and a 3:1 molar excess of a 1.0 kb insert is 0.0455 pmol, which is 30 ng. For a 300 bp insert at 5:1 it is 15 ng. Many failed assemblies are a mass ratio used where a molar ratio was intended, and the small fragment is then in vast molar excess or deficit.

The control set for a Gibson or Golden Gate reaction. A positive transformation control — 10 pg of pUC19 — which reports the competence of the cells independently of your assembly; a negative control of linearised vector with no insert, which counts the colonies that arise from uncut vector, self-ligation and carry-over; and the assembly itself. With these three, the colony counts decide what went wrong. The expected number of colonies is the correctly assembled DNA transformed, times the efficiency of the cells, times the fraction plated. With assumed values — 50 ng vector, 10% conversion to correct circles, 10% of the reaction transformed and 10% of the recovery plated, into cells at 10^8 colonies per microgram — a good assembly gives about 5,000 colonies, the same reaction in cells at 10^6 gives 50, and a weak assembly at 0.5% conversion gives 250. The pUC19 control under the same fractions gives about 100. And background: if 1% of the vector is uncut, the vector-only control gives about 500 colonies; at 0.1% uncut, 50.

Zero colonies, in order. First look at the pUC19 plate. If it also has none, the problem is the cells, the antibiotic or the heat-shock and recovery — not your DNA. If the control grew, the assembly failed: the likely causes in order are the molar ratio and the DNA amount, an inhibitor in the DNA (column carry-over of ethanol or salt, or too much PCR product in the reaction), fragments with non-compatible or damaged ends (check the overhangs of chapter 21 and whether a 5′ end was phosphorylated where it needed to be), a toxic insert, and finally assembly enzymes that have lost activity. The control that distinguishes the last from the others is assembling a known-good pair of fragments in parallel.

Only background colonies, all empty vector. Compare the assembly plate with the vector-only control. If they are about equal, the background is the whole signal, and the vector was not fully cut or not dephosphorylated, or the digest was incomplete: gel-purify the cut vector, add DpnI to remove template plasmid after PCR, and use a counterselectable marker in the vector's stuffer such as *ccdB* so that uncut vector kills the cell. If the assembly plate has many more colonies but all test as empty vector by colony PCR, the insert is joining but not in the intended way — an overhang that allows self-ligation, a vector that religates through a compatible end, or a bad insert that is not in the reaction. Colony PCR with one primer in the backbone and one in the insert gives a band only when the insert is present and in the expected orientation, with size equal to the distance from each primer to the junction; a missing band is therefore ambiguous, between empty vector and wrong orientation, which is a reason to pair it with a second primer set.

What a Sanger trace adds. A `.ab1` file holds the chromatogram — the four fluorescence traces behind the base calls. Read it by eye for what the calls hide: overlapping double peaks from the point of a mixed colony or a heterozygous population, a trace that falls off in quality after a repeat, poor quality in the first thirty or so bases and past about 800 to 1,000, and the quality values themselves. A gel confirms size; the trace confirms sequence and shows you whether the colony was pure.

Biosafety, and the conventions you will be asked for

Read your institution's framework before you order anything, because you will meet an institutional biosafety committee in your first month. In outline: biosafety level 1 covers well-characterised agents not known to cause disease in healthy adults, with standard microbiological practice and an open bench; level 2 covers agents of moderate hazard, with restricted access, biohazard signage, protective equipment, decontamination by autoclaving and safety cabinets for procedures that make aerosols. Most laboratory *E. coli* and *S. cerevisiae* work is level 1. The committee reviews recombinant and genetically modified organism work before it starts — the host, the inserted genes, whether the organism can survive outside the laboratory, and the containment and waste route. The national framework differs by country (in Korea, the Living Modified Organisms Act is the relevant statute — check the current text and your institution's implementation), and the hazardous cases are the ones you should raise before you plan them: a gene encoding a toxin, a pathway that makes a controlled compound, and any work that could change an organism's host range.

The data conventions that save you a year: a **naming scheme** that encodes type and a serial number — `S0001` for strains, `p0001` for plasmids, `P0001` for plates — with the serial never reused; an **inventory table** with one row per physical item and columns for ID, parent ID, genotype, antibiotic, date, person, freezer, rack, box and position; an **electronic notebook** entry for every experiment, with the IDs it used; and a rule for **where raw data live**, immutable and backed up, with analysis reading from it and never writing to it. Then apply it retrospectively to everything you have built, which is the exercise that shows how much of what you "know" about your strains is in your memory rather than your records.

Plasmid loss	$p' = p(1-l)/[p(1-l) + (1-p+pl)(1+s)]$
Dosage by position	$\text{copies} = 2^{(C/\tau)(1-x)}$; 4 → 1 at τ = 20 min, C = 40 min
Phred	$Q = -10 \log_{10} P$; ASCII = Q + 33; Q30 = 0.1%

Coverage	`c = N·L/G; uncovered fraction e^(−c)`
Call probability	`P(≥3 reads) binomial(depth, f); 59% at 30× for f = 10%`
Moles	`pmol = ng/(0.66 × bp)`
Colonies	`correct DNA (µg) × efficiency × fraction transformed × fraction plated`

Your Golden Gate gave 12 colonies. The vector-only control gave 9, and the pUC19 control gave about 100. Colony PCR on the 12 shows one with an insert. What do you conclude, and what do you do?

The cells are fine — the pUC19 control gave its expected 100 — and the assembly is mostly background: nine of the twelve colonies are what uncut or religated vector gives with no insert at all, so the assembly contributed three colonies and one confirmed correct clone. The signal-to-background is poor and it is the vector that is at fault. First step is to cut the vector more completely: increase digestion time and enzyme, gel-purify the cut backbone, and check that the stuffer fragment is removed by running the digest on a gel. Second, change the vector or the design so that uncut vector cannot give colonies — a *ccdB* stuffer in a *ccdB*-sensitive host. Third, check the molar ratio: if the insert is in low molar excess, empty vector religation wins.

The one correct clone is not wasted: sequence it, and if it is right you have your construct and the question of efficiency becomes a question of cost rather than success. But I would not scale this up before the background is under control, because the next construct, in a bad week, will give twelve colonies with none correct.

Three things people get wrong

- ‘The plasmid is stable, we grew it for ten generations.’ Production is thirty to forty, and a burden of a few per cent compounds a loss rate that looked harmless at ten.
- ‘More depth always finds more.’ It finds rarer real variants and more false ones; the caller's filters, not the depth alone, decide the table.
- ‘Zero colonies means the assembly failed.’ Look at pUC19 first; if the control also failed, the DNA is not the problem.

Self-check — answer on paper first

1. A plasmid is lost at 5×10^{-4} per division and plasmid-free cells grow 4% faster. Roughly what fraction carries it after 50 generations, and what does that imply for a production run?

Iterating the recursion, about 92%: the effective net loss rate per generation is a few times 10^{-3} once the growth advantage acts on the lost fraction, so over fifty generations the carrying fraction falls by about eight per cent, and by a hundred it is near 60%. That is tolerable for a short run and not for a long continuous one, where the same effect continues to compound; and a plasmid carrying a burdensome pathway has a larger s than this, so the number would be worse. The check is to plate the culture with and without selection at the end of the run.

2. You sequence a clonal isolate at 40× and a mixed population at 40×. For each, what can you call, and what can you not?

At 40× the clonal isolate's real variants are at about 100% frequency and are called with certainty, so you can build a reliable mutation table; the only concern is false calls from mapping artefacts in repeats. In the population, a variant at 10% frequency has about a 78% chance of reaching three supporting reads and one at 5% about 32%, so you cannot see minor subpopulations reliably and a negative is uninformative below about 10 to 15%. For the population you want 100× or more and polymorphism mode — and then a stricter filter, since the noise grows with depth.

3. Explain why the serine integrase's irreversibility is useful in a production strain and what you would have to supply to reverse it.

The product sites attL and attR are not recognised by the integrase alone, so once the cassette is in it will not excise during the fermentation, whatever the cell does with the integrase, and the integrase can be lost with its helper plasmid without risk. To reverse it you need the phage's recombination directionality factor, which changes the integrase's preference toward attL × attR and allows excision — useful if you want to remove a cassette deliberately, and a reason not to express it by accident.

What to open a book for. A review of serine integrases and landing pads for the mechanism and the toolkit; the breseq documentation and Deatherage & Barrick, Methods Mol. Biol. 1151:165 (2014); the NCBI Sequence Read Archive for the dataset; and Addgene's protocols on colony PCR and cloning troubleshooting, which are short and good. Part 7 chapters 21 to 25 are the chapters to have open beside the wet-lab section.

RNA-seq and differential expression: what a count matrix assumes

A count matrix looks like data. It is data that has already passed through five assumptions, one of which — that most genes do not change — a production strain can violate by design. This chapter shows each assumption failing, on data where the right answer is known.

From reads to counts, and what is baked in

A bacterial RNA-seq experiment turns RNA into cDNA, sequences it, and asks for each gene how many reads came from it. Three choices made before any statistics determine what the numbers can mean. **rRNA depletion**: ribosomal RNA is most of the total RNA in a bacterial cell, so without removal most reads are wasted on it and the mRNA that is left is sampled shallowly; with removal, the residual rRNA fraction varies between libraries, which becomes a source of technical variation. **Strandedness**: a stranded protocol records which strand a read came from, which separates a gene from an antisense transcript or from a gene on the opposite strand overlapping it — important in the compact, overlapping gene arrangements of bacteria and for detecting the small regulatory RNAs of chapter 27. **Depth**: with about 4,500 genes, 5 million reads of which perhaps 60% are mRNA gives a mean of roughly 650 reads per gene, but the distribution is wide, and the median gene has far fewer, so the genes you care about at low expression may have counts in the tens.

Quantification turns reads into counts. **Alignment-based** methods map reads to the genome and count those overlapping each gene's annotation. **Selective-alignment** methods such as salmon assign reads to transcripts directly, handle ambiguous reads probabilistically, and correct for fragment length and sequence-composition bias; in a bacterium without splicing the advantage is smaller than in a eukaryote, but it helps with overlapping genes and operons. Either way the count matrix hides assumptions you should be able to write down: that the annotation is complete and correct; that reads that map to more than one place have been handled by a stated rule; that a read is counted for the gene it overlaps and not for a longer transcript it belongs to; and that the library is a random sample of the RNA, which is not true of any protocol with a size-selection or amplification step.

Why raw counts lie: composition

The central fact of RNA-seq is that you sequence a fixed number of reads in total, so the counts are *relative*: a gene's count is proportional to its share of the library, not to its absolute amount in the cell. If one gene takes a larger share, every other gene's share must fall, even if their absolute levels did not change.

Demonstrate it, with a simulated experiment where the answer is known. Take 5,000 genes with broad expression, and two conditions with three replicates each at 5 million reads. In the second condition a single gene — your pathway — is 20% of all mRNA; nothing else changes. Compare each gene's share of the library between the two conditions with simple library-size scaling:

```
apparent log₂ fold change of every unchanged gene = log₂(0.80 / 0.9999) = −0.32
```

That is what the simulation gives: a median apparent change of −0.323, and **86% of the genes that did not change have an apparent |log₂FC| above 0.25**. A DESeq2-style *median-of-ratios* normalisation estimates a size factor for each library as the median across genes of the ratio of the gene's count to its geometric mean over all libraries. Because the median gene is unchanged, the size factor ignores the one outlier gene and scales the library by the right amount: the unchanged genes now have a median log₂FC of 0.000 and only 3% exceed 0.25, which is the noise.

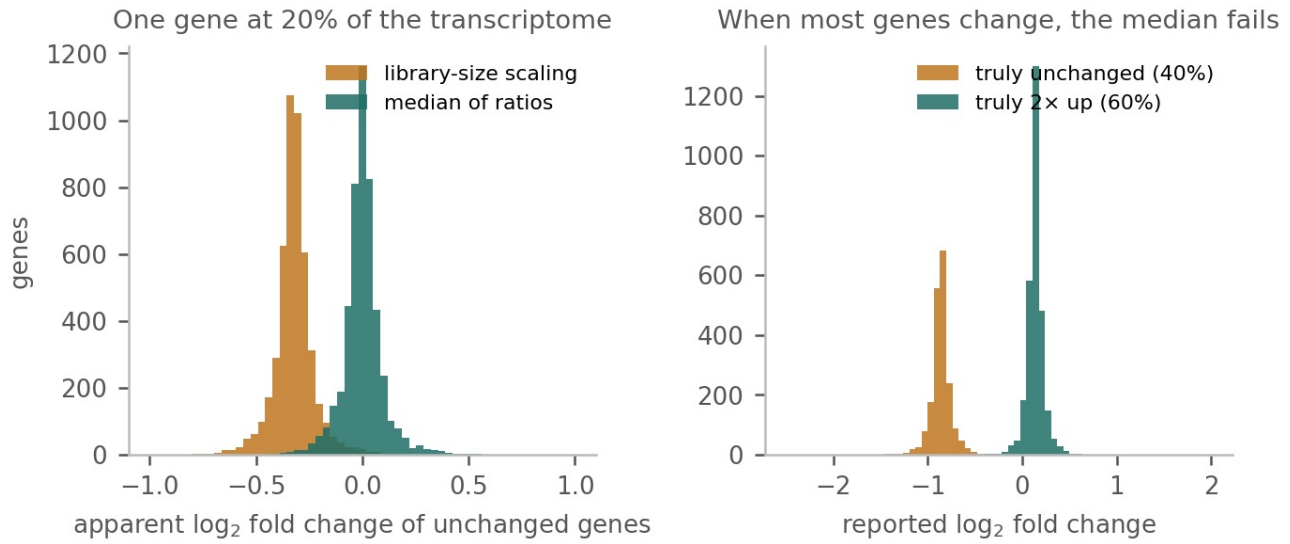


Figure 33.1 Left: apparent log₂ fold changes of unchanged genes with library-size scaling and with median-of-ratios, when one gene takes 20% of the mRNA. Right: reported log₂ fold changes after median-of-ratios normalisation when 60% of genes are truly doubled. Simulated with 5,000 genes, 5 million reads per library and three replicates; all parameters assumed.

When the assumption fails. Median-of-ratios assumes that most genes do not change — the median gene is a reliable internal standard. A production strain can violate that. Suppose 60% of all genes truly go up two-fold, as in a global shift between growth phases or a stress response. The median gene is now one of the changed ones, so the size factor is set by them and rescales the library so that *they* look unchanged. The result in the simulation is that the truly-doubled genes are reported at a median log₂FC of +0.14 — essentially unchanged — and the genes that truly did not change are reported at −0.86, nearly two-fold *down*, with 98% of them past a 1.5-fold cut. The experiment has inverted the biology.

The tracker's specific warning for production strains follows from the first case. A pathway expressed to 20% of the transcriptome is a strongly asymmetric composition shift, and while median-of-ratios handles one dominant gene, it does not handle a pathway-driven global change, and the standard defence is a spike-in: a known quantity of an external RNA added in proportion to cell number, which gives a normaliser independent of the cell's own composition. If your question is about a global shift, say so in the design and add the spike-in; if it is not, check that the median-ratio assumption holds by looking at the distribution of fold changes, which should be centred on zero.

Differential expression: what the model does

DESeq2 and edgeR model each gene's counts as negative binomial, in which the variance exceeds the mean by a term that grows with the square of the mean:

```
Var(count) = μ + α·μ²        α = dispersion, the squared coefficient of variation of the biological variability
```

At low counts the Poisson term μ dominates; at high counts the biological variability α μ² dominates, which is why a highly expressed gene is not measured arbitrarily precisely however deep you sequence. The practical consequence is a floor on the precision of a log fold change. For n replicates per condition, the standard error of the log fold change is approximately $\sqrt{[(2/n)(1/\mu + \alpha)]}$, and as μ grows it tends to $\sqrt{(2\alpha/n)}$, set by biology alone. With α = 0.1 and n = 3 that floor is 0.26 natural-log units, or 0.37 in log₂, however many reads you buy.

The second thing DESeq2 does is the reason n = 3 is workable at all. With three replicates the per-gene dispersion is badly estimated, so the method fits a trend of dispersion against mean across all genes and *shrinks* each gene's estimate toward the trend — borrowing information across 4,500 genes to stabilise each one. Fold changes can then also be shrunk, which reduces the inflation of large estimates in low-count genes. This works, but only so far: it is a statistical rescue of an experiment that is under-replicated, and the arithmetic of what three replicates can see is sobering.

What n = 3 can and cannot find

With the delta-method standard error above, the power to call a given fold change at a critical value corresponding to α = 10^{-3} (a stand-in for an FDR-adjusted threshold on thousands of genes; the critical z is 3.29) is:

Mean count	2-fold: n = 3	n = 6	n = 12	1.5-fold: n = 3	n = 6	n = 12
10	0.08	0.27	0.69	0.01	0.04	0.14
100	0.23	0.63	0.97	0.04	0.12	0.38

1,000	0.27	0.69	0.98	0.04	0.14	0.43

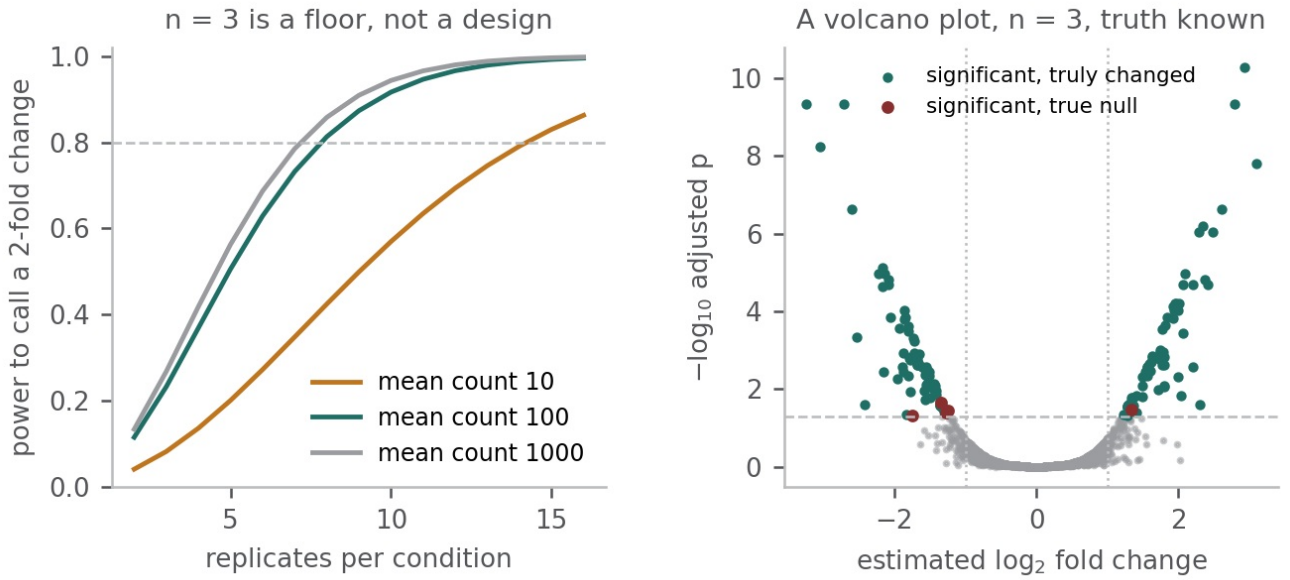


Figure 33.2 Left: power to call a 2-fold change against the number of replicates at three expression levels, from the delta-method standard error with a dispersion of 0.1 and a critical z of 3.29. Right: a volcano plot of a simulated n = 3 experiment in which 10% of genes truly change, coloured by truth. Computed for this module; the dispersion and the effect-size distribution are assumptions.

So at three replicates and a typical mean of a hundred, a real two-fold change is detected 23% of the time and a real 1.5-fold change 4%; it takes about six replicates to reach 63% and twelve to reach 97% for the two-fold case. "n = 3 is the practical floor" is true in that three is the minimum for a variance estimate, and false in that it is an adequate design: a three-replicate experiment is a survey for large effects, and a failure to see a modest one is no evidence that it is absent — the part 2 chapter 17 argument, now with numbers for RNA-seq.

Multiple testing, and what a threshold hides

A genome-wide test is thousands of tests at once. In the simulation, 4,524 genes had no true change, and with an *unadjusted* p < 0.05 there were 484 calls, of which 234 — **48%** — were true nulls. That is about 5% of the 4,524 nulls. A list of 6,000 genes tested at the same cutoff with no true changes at all would return 300 "hits". The Benjamini–Hochberg procedure controls the expected fraction of false discoveries among the calls instead: sort the p-values, and call every gene whose p-value lies below (rank/G) × q.

Now vary the cutoffs on the same simulated data, at two replicate numbers, and read off what changes. Of 5,000 genes, 10% truly changed, with true $\log_2$ changes drawn from a normal distribution of standard deviation 1 — many of them small.

Replicates	Adjusted p below	Fold-change cut (|log₂|)	Genes called	True nulls among them	True changes found
3	0.10	none, 0.585 or 1	165	7.9%	32%
3	0.05	none, 0.585 or 1	133	3.8%	27%
3	0.01	none, 0.585 or 1	93	0%	20%
8	0.05	none or 0.585	247	5.3%	49%
8	0.05	1	170	0.6%	36%
8	0.01	1	163	0.6%	34%

Two results are worth the table. **At n = 3 the fold-change cutoff does nothing**: every gene that reaches adjusted p < 0.05 already has an estimated |log₂FC| of at least 1.23, because with a standard error near 0.4 only a large estimated change can be significant. So "p < 0.05 and 2-fold" is the same list as "p < 0.05", and quoting the two-fold cutoff gives the impression of a stringency you did not apply. At n = 8 the cutoff matters — it removes 77 genes and drops the false fraction from 5.3% to 0.6% — and it also discards true changes that were small, so it is a real trade, which is what the tracker means by "a two-fold cutoff is a convention rather than a biological threshold": a gene 1.5-fold up that is the entry step of your pathway matters more than a ten-fold change in a stress gene, and no cutoff knows that.

The second result is the winner's curse. Among the true changes called at n = 3, the mean estimated |log₂FC| was

1.21 times the true one; at n = 8, 1.05. Genes are selected for being significant, and at low precision the genes that cross the line are disproportionately those whose noise pushed them across, so their estimates are inflated. That is exactly the bias that shrinkage of fold changes corrects, and it is why a paper's list of "top genes by fold change" from a three-replicate experiment overstates its effects.

Defend a pair of thresholds in a sentence, as the week asks: *adjusted $p < 0.05$ to hold the expected false discoveries to about one in twenty of the calls, and no fold-change cutoff at $n = 3$ because significance already implies a large change, with a note that the experiment is powered for changes above about two-fold.*

Running it, and reading it as a biologist

The analysis in R, not run here: build a `DESeqDataSet` from the count matrix and a sample table with `DESeqDataSetFromMatrix(countData, colData, design = ~ condition)`, run `DESeq(dds)`, extract `results(dds, alpha = 0.05)`, and apply `lfcShrink` before ranking by fold change. In Python, `pydeseq2` implements the same model. Public count matrices for a strain comparison are on the Gene Expression Omnibus; reanalyse one and record how many genes pass at your thresholds and what the volcano plot's shape says about the replication.

Transcript is not flux

Three reasons a transcript level can rise while flux does not, each quantifiable from earlier parts. **Translation and turnover**: part 1 chapter 8's expression equation puts protein at transcription × initiation ÷ (degradation + dilution), so a doubled transcript can give an unchanged or only modestly changed protein if initiation or degradation changed. **Enzyme activity**: the enzyme can be present and inactive — misfolded, missing a cofactor, post-translationally modified or inhibited — the point of chapter 22's band-is-not-activity warning. **Metabolite control**: even a fully active, doubled enzyme changes flux only by its control coefficient — for an enzyme with C = 0.10, doubling the activity buys at most 5% (chapter 30), and for one saturated by its substrate the gain is smaller still. A transcriptome therefore tells you what the cell regulated, which is useful for diagnosing a stress response or a mis-regulated promoter, and it is a weak proxy for what the pathway did.

Take your top differentially expressed genes and say what you would measure next: for each, the protein (by Western blot or proteomics), then the activity in a lysate under saturating substrate, then — if it matters — the intracellular substrate concentration against K_M to decide whether the enzyme is in its first-order regime, and finally the flux, by a carbon balance or ^{13}C labelling (week 43). That sequence is the week 51 argument in the order you would run it.

Composition shift	apparent $\log_2 FC = \log_2[(1 - x)/(1 - x_0)]$ for a gene taking share x
Median of ratios	size factor = median over genes of count/geometric mean; fails if most genes change
Negative binomial	$\mathrm{Var} = \mu + \alpha\mu^2$; $\mathrm{SE}(\ln FC) \approx \sqrt{[(2/n)(1/\mu + \alpha)]}$
Power, n = 3	0.23 for a 2-fold change at mean 100, $\alpha = 0.1$, $z_{crit} = 3.29$
Unadjusted false calls	5% of the true nulls — 234 of 4,524 here
Winner's curse	estimated/true = 1.21 at n = 3, 1.05 at n = 8, among called genes

A colleague compares an engineered strain expressing a pathway at roughly 25% of its mRNA against the parent, n = 3, and reports 600 genes "significantly down at least two-fold". How do you read that?

With a pathway taking about a quarter of the transcriptome, naive library-size scaling would make every other gene look about 25% lower — a $\log_2$ shift of about -0.4 — so a large share of "down" calls could be composition, not biology. First check which normalisation was used and look at the distribution of $\log_2$ fold changes for the bulk of genes: if it is centred on -0.4, normalisation did not correct it; if it is centred on zero, median-of-ratios did, and the 600 are more credible. Second, the genes truly downregulated by a heavy burden are exactly what you expect — ribosomal proteins and growth-related genes fall under a stringent-response-like state — so the biology is plausible, but plausible is not demonstrated.

Third, at n = 3 the two-fold cutoff is redundant with significance, as the table shows, so the claim of "at least two-fold" is carried by the significance threshold alone, and the estimated fold changes of the significant genes are inflated by about 20%. What I would ask for is the $\log_2 FC$ histogram, the normalisation, the adjusted-p cutoff, and whether a spike-in or a second normalisation method was used as a check — and, if the question is global, a spike-in in the next run.

Three things people get wrong

- ‘Counts are abundance.’ They are shares of a fixed library; one dominant gene lowers every other gene's apparent level until normalisation is checked.
- ‘$p < 0.05$ and two-fold is a strict filter.’ At three replicates significance already implies a large change, so the second

condition adds nothing.

- ‘The gene is upregulated, so the pathway flux rose.’ Two lossy steps and a control coefficient sit between them.

Self-check — answer on paper first

1. One gene takes 40% of the mRNA in condition B and 1% in A. With library-size scaling, what apparent $\log_2$ change does an unchanged gene show?

Its share in B is multiplied by $(1 - 0.40)/(1 - 0.01) = 0.606$, so $\log_2(0.606) = -0.72$, an apparent 1.65-fold decrease in every unchanged gene. Median-of-ratios would remove this, provided most genes are in fact unchanged; with a spike-in the correction does not depend on that assumption.

2. You have a budget for either 3 replicates of each of 4 strains or 6 of each of 2. Which and why?

Six of two, if the question is a pairwise comparison, because power for a two-fold change at a mean of 100 rises from 23% to 63% on doubling the replicates, whereas the four-strain design buys breadth at a power below a quarter. If the question needs all four strains, a reduced design — a reference and three test strains, with more replicates of the reference — is better than equal n = 3 throughout, since every comparison involves the reference and its precision is shared.

3. Your 3-replicate experiment returns 20 genes at adjusted $p < 0.05$, and you expected hundreds. What do you do before concluding the strain is unchanged?

Compute what the design could have seen: at n = 3 and a dispersion of 0.1, a real two-fold change at a mean of 100 is found about a quarter of the time and a 1.5-fold change about 4%, so a short list is what a weak experiment returns whether or not there is biology. Look at the dispersion estimates — a high one means the replicates disagreed, which is an experimental problem to fix — and at the raw fold-change distribution, where a shift of many genes by small amounts is invisible to gene-by-gene testing and visible to a pathway-level analysis. Then replicate to at least six.

What to open a book for. Love, Huber & Anders, Genome Biol. (2014) for DESeq2, read for dispersion estimation and shrinkage rather than the software; Patro et al., Nat. Methods (2017) for salmon; the Bioconductor DESeq2 vignette, which is the best tutorial there is; and the Gene Expression Omnibus for a count matrix to reanalyse. Part 4 chapters 17 and 51 are the two chapters of this module that this chapter is built on.

The biofoundry as a data pipeline: plate plans, protocols and position effects

A biofoundry is a data pipeline with robots at some of the nodes. Most of what goes wrong is not the robot: it is an identifier that was lost, a plan whose volumes were not counted, or a plate in which position was mistaken for biology.

The cycle as a pipeline, and why identifiers come first

The design–build–test–learn cycle of part 2 chapter 11 becomes, in a foundry, a sequence of files. A **design file** lists each variant and its parts. A **build plan** turns it into a table of reactions with source wells, volumes and destination wells. An **automation protocol** executes the plan. A **test** produces plate-reader or chromatography data keyed by well. A **results table** joins the data back to the design, and the next round's design is computed from it. K-Biofoundry, which the tracker names as KRIBB's facility, and the Global Biofoundry Alliance it belongs to, exist so that groups can share the equipment, the standards and the people for this loop; what an academic group gets from membership that a single laboratory does not is throughput, standardisation and the discipline of a pipeline that someone else has already debugged.

The load-bearing component is the **identifier**. Every strain, plasmid, plate and well needs one that is created once, never reused, and carried through every file. A workable scheme: a design ID per variant (`V00417`), a plate barcode per physical plate (`PL0231`), a well address (`B07`), and a sample ID that is their concatenation (`PL0231-B07`). The results table is then a join of the design table and the data table on sample ID, and every result traces back to its design by a key rather than by someone's memory.

What goes wrong if two plates share a barcode: the join is many-to-many, so a well on one plate silently takes data from the other. Nothing errors, every row has a value, and the results are plausible and wrong — which is the worst failure a pipeline has. The defences are that barcodes are generated and checked for uniqueness, that a join asserts a one-to-one match and fails loudly if it does not, and that the plate map is stored in the same file as the data it describes.

From a variant list to a plate plan

The week's concrete exercise: a 24-variant combinatorial assembly — four promoters, three coding sequences, two terminators. The code below builds the plan and counts what the robot will need; I ran it, and the numbers in the table that follow are its output.

```
import itertools, pandas as pd
prom, cds, term = ['P1','P2','P3','P4'], ['C1','C2','C3'], ['T1','T2']
plan = pd.DataFrame(list(itertools.product(prom, cds, term)), columns=['prom','cds','term'])
plan['dest'] = [f"{'ABCDEFGH'[i//12]}{i%12+1}" for i in range(len(plan))]     # 24 of 96 wells

rxn_uL, overage, dead_uL = 10, 1.10, 10
# per 10 uL reaction: 1 backbone + 3 parts (1 uL each) + 1 buffer + 1 enzyme mix + 4 water
common = 1 + 1 + 1 + 4                                  # in the master mix: backbone, buffer, enzyme, water
mm_total = common * len(plan) * overage                 # 184.8 uL
for name, lst in (('prom', prom), ('cds', cds), ('term', term)):
    uses = len(plan) // len(lst)                        # reactions each part appears in
    print(name, uses, 'uL +', dead_uL, 'dead =', uses + dead_uL, 'uL per source well')
print('transfers', len(plan) * 4, '; fresh-tip transfers for parts', 3 * len(plan))
```

Item	Count or volume	Why
Reactions	24	4 × 3 × 2
Master mix	7 µL × 24 × 1.10 = 184.8 µL, plus dead volume	Backbone, buffer, enzyme mix and water are common to every reaction
Each promoter well	6 µL + 10 dead = 16 µL	Each appears with 3 coding sequences × 2 terminators = 6 reactions
Each coding-sequence well	8 µL + 10 dead = 18 µL	4 promoters × 2 terminators = 8 reactions
Each terminator well	12 µL + 10 dead = 22 µL	4 × 3 = 12 reactions
Source wells / destination wells	9 / 24 of 96	4 + 3 + 2 distinct parts
Transfers	96	3 parts + 1 master mix per reaction

Tips	72 for parts; 73 if one tip serves the master mix, 96 if each reaction gets its own	A fresh tip for every part avoids cross-contamination of the stock

Three things in the table repay attention. **Dead volume** dominates the small stocks: the 10 µL left at the bottom of a well that the pipette cannot reach is more than the volume you actually use of each promoter, so the stock you must prepare is 16 µL for 6 µL consumed. That ratio worsens as the experiment gets smaller and is the reason plate plans are written as a table before a protocol is. **The master mix** is where most of the volume is, and its overage is an insurance against the pipette's imprecision at the end of a long dispense; 10% is conventional and should be tested against what is actually left. And **the number of tips** is a real consumable budget — a thousand-sample screen at 96 tips per 24 reactions is four thousand tips.

What changes in a 384-well plate: the reaction volume falls, often to 2 to 5 µL, so the dead volume is a larger fraction of everything and several parts will fall below the reliable volume of a standard pipette, which typically has a minimum of about 1 µL; the source plate, if also 384-well, has smaller wells with different dead volumes and a different liquid-height-to-volume relationship; the destination layout needs a plan for the edge effect below; and evaporation, discussed next, becomes a first-order concern.

Writing the protocol, and what the simulator cannot check

The Opentrons Python API declares labware, loads pipettes, and expresses a plan as transfers. A skeleton for the plan above, which I have not run — check the labware definitions and API level against the current documentation and run it through the simulator before it goes near a robot:

```
from opentrons import protocol_api
metadata = {'protocolName': 'GG 24-variant assembly', 'apiLevel': '2.15'}

def run(protocol: protocol_api.ProtocolContext):
    tips  = protocol.load_labware('opentrons_96_tiprack_20ul', 1)
    src   = protocol.load_labware('nest_96_wellplate_100ul_pcr_full_skirt', 2)    # parts
    dest  = protocol.load_labware('nest_96_wellplate_100ul_pcr_full_skirt', 3)    # reactions
    mm    = protocol.load_labware('opentrons_24_tuberack_nest_1.5ml_snapcap', 4)  # master mix
    p20   = protocol.load_instrument('p20_single_gen2', 'right', tip_racks=[tips])
    # plan: a table of (source well, destination well, volume) generated from the pandas plan above
    for part_well, dest_well, vol in transfers:
        p20.transfer(vol, src[part_well], dest[dest_well], new_tip='always')
```

The simulator checks that the protocol parses, that labware exists, that every transfer is geometrically possible and that you have enough tips. It cannot verify what only the machine does: **the calibration** of the deck positions, which is the responsibility of whoever calibrated the robot; **liquid class**, the way a viscous master mix with glycerol-containing enzyme aspirates differently from water and needs a slower speed and a delay; **tip behaviour**, whether tips seat and eject cleanly, and whether a droplet clings; **evaporation** during a long run; and **real liquid levels**, so that a source well that has run low is detected rather than aspirated dry. Each needs a named owner, and each is checked by a test run with a dye or a gravimetric check rather than by trusting the plan.

The PyLabRobot paper's argument is the software-engineering one: a protocol written against one vendor's API is locked to that vendor's machine, and a hardware-agnostic layer lets the same script drive different robots. The problem it solves is the portability of a protocol across a foundry's mixed fleet, and it reflects a view that protocols should be treated as versioned code — reviewed, tested, tagged — not as a file that lives on one laptop.

Miniaturisation, and what it saves and costs

For a 384-variant library with a reaction in which one tenth of the volume is enzyme mix and one tenth is buffer — so 2 µL of the 10 µL is expensive reagent — the totals are:

Reaction volume	Total reaction volume	Enzyme + buffer for 384	Saving versus 10 µL
10 µL	3,840 µL	768 µL	1×
2 µL	768 µL	154 µL	5×
500 nL	192 µL	38 µL	20×

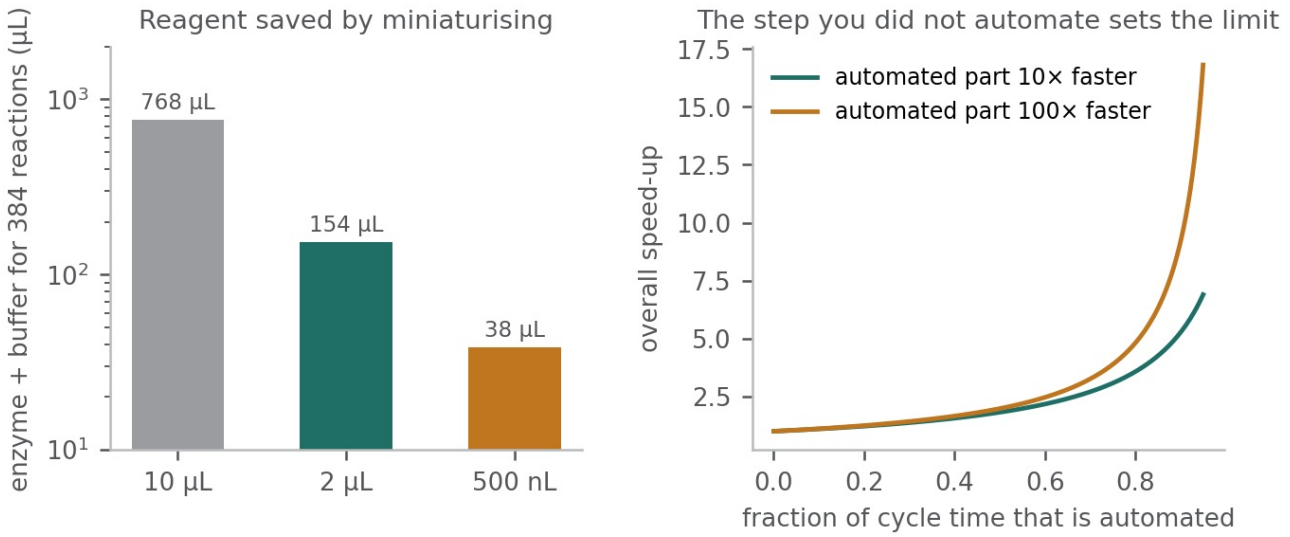


Figure 34.1 Left: enzyme and buffer needed for 384 reactions at three scales, on a log scale. Right: overall speed-up of a workflow against the fraction of its time that is automated, for an automated step that is ten and a hundred times faster than manual — Amdahl's law. Computed for this module.

The reagent saving is real and proportional to volume. What it costs is **evaporation**. A well with a fixed opening loses liquid at a roughly constant rate in volume per hour, so the fractional loss goes as one over the volume. If an open well loses 0.1 µL an hour — an assumed figure that depends on humidity, temperature and the seal — a 10 µL reaction loses 1% an hour, a 2 µL reaction 5% and a 500 nL reaction 20%. At that point concentration changes over a run and varies by position, which is the next section's subject. The mitigations are the ones chemical engineers know: seal the plate, work at a low temperature or high humidity, keep the plate on the deck as short a time as possible, and put an inert oil overlay on very small volumes.

The steps that cannot be miniaturised are the ones that involve living cells or space: transformation into cells, plating and colony picking, and growth in deep-well blocks. They set the throughput, and the arithmetic is Amdahl's law: if a fraction f of the cycle time is made s times faster, the overall speed-up is $1/[(1 - f) + f/s]$. If assembly set-up is 30% of the cycle time and automation makes it ten times faster, the whole cycle is only 1.37 times faster, and a hundredfold gain takes it to 1.42; the limit, with f = 0.3, is 1.43 whatever the robot does. That is the same lesson as part 2 chapter 11's foundry that builds 2,000 strains into a 400-sample screen: speed up the bottleneck, not the step that was already fast.

Plate data, honestly: the edge effect

Edge wells of a plate evaporate faster and sit at a different temperature, so in a growth or fluorescence assay they read high. The question is how badly that corrupts a ranking, and the answer is simulable. Take a 96-well plate with 36 edge and 60 inner wells, 96 variants whose true values vary by about 15% (lognormal), placed in random wells, edge wells reading 20% high, and 5% measurement noise.

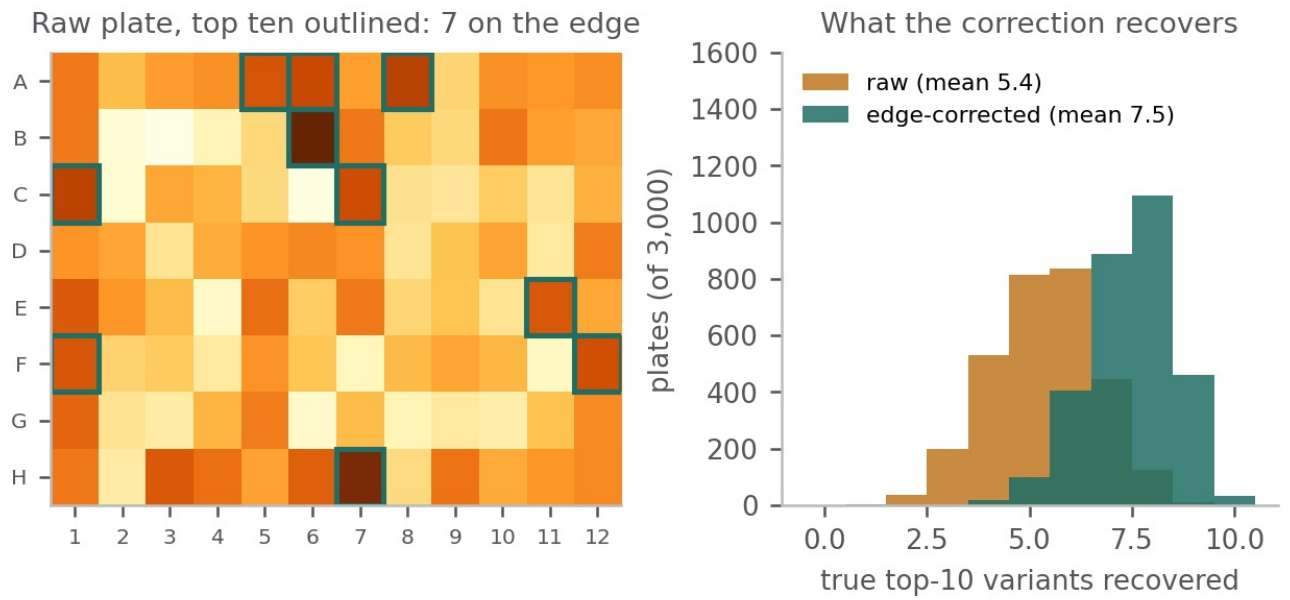


Figure 34.2 Left: one simulated raw plate with the ten highest readings outlined — seven of them are edge wells. Right: across 3,000 simulated plates, how many of the true top ten variants appear in the measured top ten, raw and after estimating the edge multiplier from the plate itself and correcting for it. Computed for this module; the 20% multiplier, 15% true spread and 5% noise are assumptions.

Averaged over 4,000 simulated plates, **8.2 of the ten highest raw readings are edge wells**, against 3.75 expected if position did not matter — the top of the ranking is dominated by evaporation. The raw ranking recovers 5.4 of the true top ten and has a Spearman correlation of 0.80 with the truth. Now estimate the multiplier from the plate itself, as the ratio of the median edge reading to the median inner reading, which works *because the variants were placed randomly* so that edge and inner wells hold statistically identical populations; the estimate comes out at 1.200, the true value. Dividing the edge readings by it recovers 7.5 of the true top ten and a correlation of 0.93.

The correction only works if the layout is randomised. In a blocked layout — one strain family filling the edge and another the inner wells — the same edge effect appears as a difference *between the families*. Simulating two identical families, one placed predominantly on the edge, gives an apparent family ratio of 1.15 where the truth is 1.00: a 15% "effect" produced by the plate, with nothing in the data to say so.

So the controls to add to the plate design are these. **Randomise the layout**, so position is not confounded with identity. **Put the same control strain in many positions** — in every row and column, and in edge and inner wells — which measures the position effect directly rather than inferring it, and also shows whether it is a simple edge multiplier or a gradient. **Leave the outer ring empty or filled with water or medium** if you can afford the wells, which turns the evaporation problem into a lost 37% of capacity. And **replicate the best candidates in a second plate** with a different layout, since a hit that survives a change of position is a hit that is not about position.

Identifier	`sample ID = plate barcode + well; joins must be one-to-one and fail loudly`
Master mix	`common µL per reaction × reactions × overage + dead volume`
Source well	`(uses of the part) × µL per use + dead volume`
Evaporation	`fractional loss ∝ 1/V at fixed open area; 1% → 5% → 20% per hour here`
Amdahl	`speed-up = 1/[(1 − f) + f/s]; limit 1/(1 − f)`
Edge multiplier	`median(edge)/median(inner), valid only for a randomised layout`

A foundry run of 960 variants in ten plates shows that the top hits cluster in columns 1 and 12. The group wants to take the top twenty into the fermenter. What do you advise?

That the clustering at the plate edges is the signature of an evaporation effect, so before anything goes into a fermenter the ranking needs to be separated from position. Check the layout first: if variants were placed in a design file order rather than randomly, position and identity are confounded and the correction in the chapter does not apply — the edge wells hold a particular subset. If the layout was randomised, estimate the edge multiplier per plate from the data and correct, then re-rank; the simulation says the correction recovers roughly two more of the true top ten, from 5.4 to 7.5 of 10.

In either case do a validation round before the fermenter: re-pick the top fifty by the *corrected* ranking, re-test them in a new plate layout with a control strain in every row and column, and in duplicate, and take forward only those that hold their position. That costs one more plate cycle and removes the class of hit that was a property of the plate. The fermenter, by contrast, costs weeks, and the lesson of part 2 chapter 17 is that the right time to find a false positive is before it is expensive.

Three things people get wrong

- ‘The robot is the risk.’ The risk is an identifier, a join, a dead volume or a position effect — the pipeline around the robot.
- ‘The simulator passed, so the protocol works.’ It checks geometry and tip counts; calibration, viscosity, tip behaviour and evaporation are only found on the machine.
- ‘Automating assembly makes the cycle ten times faster.’ If it is 30% of the cycle, it makes it 1.37 times faster; the unautomated step sets the ceiling.

Self-check — answer on paper first

1. Compute the source volume of each part and the master-mix volume for a 4 × 4 × 3 = 48-variant assembly in 10 µL reactions with 10% overage and 10 µL dead volume. What is the first thing that limits you?

Each of the four promoters appears in 4 × 3 = 12 reactions, so 12 + 10 = 22 µL; each of the four coding sequences in 4 × 3 = 12 reactions, 22 µL; each of the three terminators in 4 × 4 = 16 reactions, so 16 + 10 = 26 µL. The master mix is 7 µL × 48 × 1.10 = 369.6 µL plus dead volume, which is far more than a PCR-plate well holds, so it must go into a 1.5 mL tube — which the tube rack in the skeleton provides. The first limit is the tips: 48 × 3 = 144 fresh-tip transfers for the parts is more than one 96-tip rack, so the deck needs two, not the part wells or the master mix.

2. A fluorescence plate shows its highest values in the corner wells. Give two explanations that are not biology, and one control that separates them from a real effect.

Evaporation raising the concentration of the fluorophore and the cells in edge wells, and a temperature or aeration gradient from the plate's position in the incubator or reader. A control strain placed in a grid of positions across the plate gives a position map: if the control reads high in the corners, the effect is position, and it can be measured and divided out; if it reads uniform, the corner values are properties of the strains placed there.

3. Why does miniaturising from 10 µL to 500 nL save twenty-fold in reagent but not twenty-fold in time?

Reagent scales with volume, but time is set by the slowest step in the cycle, and the steps that cannot be miniaturised — transformation, plating, colony picking, growth, and the assay read — keep their duration. By Amdahl's law, if the assembly set-up is 30% of the cycle time, even an infinite speed-up of it gives a 1.43× overall gain. And the smaller volume brings evaporation and dispensing-accuracy problems that add repeats and controls, which can erode the gain further.

What to open a book for. K-Biofoundry and the Global Biofoundry Alliance sites for what a foundry offers; the Opentrons Python API documentation and the PyLabRobot paper for protocols as code; Seong et al., Trends Biotechnol. (2026), the biofoundry workflow section; and pandas' "10 minutes to pandas" for the joins. Part 2 chapter 11 is the chapter whose bottleneck argument this one puts numbers to.

Closing notes to part 9

The three weeks in one page

Week 32. A plasmid lost at 10^{-3} per division with a 5% burden is below half carrying by generation 79, and at 10^{-2} by generation 33 — inside a seed train. Serine integrases make an irreversible insertion into a landing pad, and the pad's position sets dose: 4× origin-to-terminus at a 20-minute doubling time. Q30 is a 0.1% error; at 100× and independent 10^{-3} errors, three-read agreement still yields about 83 false calls per genome; and a 10%-frequency variant needs about 60× to be called 95% of the time. The colony counts of the pUC19 and vector-only controls decide whether a failed assembly is the cells, the vector or the DNA.

Week 33. One gene at 20% of the mRNA makes every unchanged gene look 0.32 $\log_2$ down until median-of-ratios corrects it, and if most genes change the same method inverts the result. At n = 3, power for a two-fold change at a mean of 100 is 0.23, a fold-change cutoff is redundant with significance, and significant genes' fold changes are inflated by about 20%. Unadjusted $p < 0.05$ returned 234 false calls of 484. A transcript is a weak proxy for flux.

Week 34. A 24-variant plan needs 184.8 µL of master mix, 16 to 22 µL per part well because of dead volume, and 96 transfers. Miniaturising to 500 nL saves 20× in reagent and costs 20% an hour in evaporation. Automating 30% of a cycle by ten times speeds it 1.37×. Edge wells filled 8.2 of the top ten raw readings, and a correction estimated from the plate itself recovered 7.5 of the true top ten against 5.4 — but only for a randomised layout.

On the numbers in this part

The sequencing-depth, false-call, plasmid-retention, dosage and colony-count figures are analytic or iterated recursions with assumptions stated beside them — the loss rates, the burden, the 40-minute replication time, the transformation fractions. The RNA-seq results and the edge-effect results are simulations with known truth: 5,000 genes, a lognormal mean-expression distribution, a dispersion of 0.1, a 10% truly changed set for the threshold study; and a 96-well plate with a 20% edge multiplier. The power numbers use a delta-method standard error rather than a full negative-binomial fit, so they are good to perhaps a few percentage points and should be rerun with real data. Nothing here was produced by breseq, DESeq2, salmon or the Opentrons simulator, which I could not run; the commands and code in those sections are marked not run, and should be checked against current documentation before use.

What remains

Written so far: weeks 1 to 34 and 45 to 61 — 51 of the tracker's 66. Remaining: weeks 35 to 44 and 62 to 66. The next block is the modelling software: constraint-based modelling and genome-scale models in COBRApy, reconstructing a model, and the MATLAB toolbox (weeks 35 to 38), where the part 2 chapter 13 material becomes code you run. After that, the design-algorithm, machine-learning and fluxomics weeks with the process weeks (39 to 44), and then the five professional weeks. I will take weeks 35 to 38 next.